

IAM-95 LAB: Terraform IAM Provisioning — Groups, Policies, Boundaries, Bindings

Series: IAM — IAM Administration | **Reference:** 95 — Terraform IAM Provisioning LAB | **Created:** July 2026 | **Last Updated:** 07/16/2026

Overview

Hands-on lab: provision a complete persona + team IAM model with **Terraform** using the [Terraform-IAM-MZ-Boundries](#) modules. Every command and expected output in this lab comes from a real run, verified live against the Account Management API on 07/16/2026.

This is one of **three provisioning flavors** — pick the one that fits your workflow:

Flavor	Notebook	Best for
curl / shell	IAM-12	One-off scripts, CI glue
Terraform (this lab)	IAM-95	Declarative IaC, drift detection, team review via plan/apply
Python	IAM-96	Programmatic onboarding, integration into portals/tooling

What you will build (the BPN pattern from IAM-05 / IAM-10):

```
Persona groups (Standard / Power / Admin)
└─ account-level persona policies, bound 1:1

Team groups – driven entirely by ONE `teams` variable
└─ Policy binding (scoped to ONE environment)
    └─ Policy: ONE shared parameterized template ← created once, reused
    by every team
        |      ALLOW storage:logs:read WHERE storage:dt.security_context IN
    ("${bindParam:team}");
    └─ Boundary: one per team (defense-in-depth)
        storage:dt.security_context IN ("&lt;team-context&gt;");
```

Time: 30–45 minutes | **Difficulty:** Intermediate | **Cost:** No consumption impact (IAM objects only)

Table of Contents

1. [Get the Project and Inspect the Modules](#)
2. [Configure Credentials and Teams](#)
3. [Initialize and Plan](#)
4. [Apply](#)
5. [Validate](#)
6. [The Payoff: Onboard a Team with Three Lines](#)
7. [Clean Up](#)
8. [Troubleshooting](#)
9. [Validation Checklist](#)
10. [References](#)

Prerequisites

Requirement	Details
Terraform	>= 1.3 (<code>terraform version</code>)

Requirement	Details
Project	<code>git clone git@github.com:timstewart-dynatrace/Terraform-IAM-MZ-Boundries.git</code>
Account UUID	Account Management → Account settings
OAuth client	Account Management → Identity & access management → OAuth clients
OAuth scopes	<code>account-idm-read account-idm-write iam:policies:read iam:policies:write iam:bindings:read iam:bindings:write iam:boundaries:read iam:boundaries:write account-env-read</code>
Prior reading	IAM-04 (policy authoring), IAM-05 (boundary design), IAM-10 (templated policies)

⚠ Safety rails (do not skip):

1. Run this lab only against a **sanctioned test tenant** — confirm the environment ID with your team before executing.
2. Keep `name_prefix` set (e.g. `TFTEST-`) so every created object is unmistakably a lab artifact.
3. Keep `binding_environment_id` set. IAM groups/policies/boundaries are **account-level** objects; the binding is what grants access, and environment-scoping it confines the lab's effect to one environment.

Step 1 — Get the Project and Inspect the Modules

```
git clone git@github.com:timstewart-dynatrace/Terraform-IAM-MZ-Boundries.git
cd Terraform-IAM-MZ-Boundries/examples/complete
```

The example composes four modules — each maps 1:1 to an IAM object type:

Module	Resource
<code>modules/iam_groups</code>	<code>dynatrace_iam_group</code>
<code>modules/iam_policies</code>	<code>dynatrace_iam_policy</code> (account level)
<code>modules/policy_boundaries</code>	<code>dynatrace_iam_policy_boundary</code>

Module	Resource
modules/policy_bindings	dynatrace_iam_policy_bindings_v2 (parameters + boundaries)

Step 2 — Configure Credentials and Teams

```
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars` :

```
dynatrace_account_id    = "<your-account-uuid>"
dynatrace_client_id     = "dt0s02.XXXXXXXX"
dynatrace_client_secret = "dt0s02.XXXXXXXX.YYYY..."

name_prefix              = "TFTEST-"          # every object gets this prefix
binding_environment_id   = "abc12345"         # YOUR test environment ID

teams = {
  alpha = {
    name              = "Alpha"
    security_context   = "tfctest-alpha"
    management_zone    = "TFTEST-Alpha"      # optional Gen2 condition (MZ2POL
transitional)
  }
  beta = {
    name              = "Beta"
    security_context   = "tfctest-beta"      # no MZ -> pure Gen3 boundary
  }
}
```

`terraform.tfvars` contains credentials — it is gitignored. Never commit it.

Step 3 — Initialize and Plan

```
terraform init
terraform plan
```

Expected output (tail of plan):

```
Plan: 16 to add, 0 to change, 0 to destroy.
```

16 resources = 5 groups (3 personas + 2 teams) + 4 policies (3 personas + 1 shared template) + 2 boundaries + 5 bindings. Inspect the parameterized policy in the plan — the `${bindParam:team}` placeholders must appear **literally** (they are resolved per binding, not by Terraform):

```
+ statement_query = &lt;&lt;-EOT
  ALLOW environment:roles:viewer;
  ALLOW app-engine:apps:run;
  ALLOW document:documents:read;
  ALLOW storage:logs:read WHERE storage:dt.security_context IN
("${bindParam:team}");
  ...
```

Step 4 — Apply

```
terraform apply
```

Type `yes` at the prompt. **Expected output (abridged):**

```
module.iam_groups.dynatrace_iam_group.this["team_alpha"]: Creation complete
after 0s [id=00d43484-fc8a-435f-8e4f-57ea9d056d35]
module.policy_boundaries.dynatrace_iam_policy_boundary.this["team_alpha"]:
Creation complete after 0s [id=d81e7853-abd4-4159-908f-75c202432cf1]
module.iam_policies.dynatrace_iam_policy.this["team_data_access"]: Creation
complete after 1s [id=dea9e886-...#-#account#-#&lt;account-uuid&gt;]
module.policy_bindings.dynatrace_iam_policy_bindings_v2.this["team_alpha"]:
Creation complete after 1s [id=00d43484-...#-#environment#-#abc12345]

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.
```

Note the binding IDs end in `#-#environment#-#abc12345` — proof the grants are environment-scoped.

Step 5 — Validate

In the UI — Account Management → Identity & access management:

- **Groups:** search `TFTEST-` → 5 groups
- **Policies:** open `TFTEST-tpl-team-data-access` → statement shows `${bindParam:team}` placeholders; its **bindings** tab shows per-team resolved values
- **Boundaries:** `TFTEST-bnd-team-alpha` shows all three conditions; `TFTEST-bnd-team-beta` shows Gen3 conditions only

Via the API (or use the IAM-96 Python tool). **Expected binding for the Alpha team:**

```
{
  "policyUuid": "dea9e886-d49d-4989-ac15-c193e823dd00",
  "groups": ["00d43484-fc8a-435f-8e4f-57ea9d056d35"],
  "parameters": { "team": "tftest-alpha" },
  "boundaries": ["d81e7853-abd4-4159-908f-75c202432cf1"]
}
```

Expected boundary query (Alpha — with transitional MZ condition):

```
storage:dt.security_context IN ("tftest-alpha");
settings:dt.security_context IN ("tftest-alpha");
environment:management-zone IN ("TFTEST-Alpha");
```

✓ **Checkpoint:** `parameters.team` matches the team's `security_context`, and exactly one boundary UUID is attached.

Step 6 — The Payoff: Onboard a Team with Three Lines

Add to `teams` in `terraform.tfvars`:

```
gamma = {
  name           = "Gamma"
  security_context = "tftest-gamma"
}
```

```
terraform plan
```

Expected output:

```
Plan: 3 to add, 0 to change, 0 to destroy.
```

Three resources — group, boundary, binding. **The shared policy is untouched.** That is the entire value of the templated-policy pattern: teams scale without policy sprawl. Apply if you want, then continue.

Step 7 — Clean Up

```
terraform destroy
```

Expected output (tail):

```
Destroy complete! Resources: 16 destroyed.
```

(19 if you applied Gamma.) Verify in the UI that no `TFTEST-` objects remain, then delete `terraform.tfvars`.

Troubleshooting

Symptom	Cause	Fix
400 <code>Invalid permission name: ...</code> on apply	Statement uses a nonexistent permission (e.g. <code>environment:roles:admin</code> , <code>storage:spans:write</code>)	See the verified permission rules in IAM-04 and error table in IAM-12 §7
400 <code>ALLOW or DENY must be followed by a permission...</code>	Wildcard in a statement (<code>storage*:read</code>)	Wildcards are not supported — enumerate permissions
401 on plan/apply	OAuth client credentials wrong or scopes missing	Recreate client with the full scope list above
Binding created at account level unexpectedly	<code>binding_environment_id</code> unset	Set it — null means account-level bindings

Symptom	Cause	Fix
<code>\${bindParam:team}</code> resolved to nothing by Terraform	Written as <code>\${...}</code> in HCL	In HCL heredocs escape as <code>`\${bindParam:team}`</code>

Validation Checklist

- [] `terraform plan` showed 16 to add before apply
- [] Binding IDs contain `#-#environment#-#<env-id>`
- [] The shared policy statement contains literal `${bindParam:team}`
- [] Alpha's binding carries `parameters: {team: tftest-alpha}` + 1 boundary
- [] Beta's boundary has NO `environment:management-zone` line (pure Gen3)
- [] Adding a team planned exactly **3** new resources
- [] `terraform destroy` removed everything (UI search for prefix is empty)

References

- **IAM-04** Policy Authoring (verified statement syntax) · **IAM-05** Boundary Design · **IAM-10** Templated Policy Assignments · **IAM-12** API Provisioning (curl flavor) · **IAM-96** Python lab (same pattern, Python flavor)
- **MZ2POL series** — migrating off Management Zones onto this pattern
- **AUTOM-95 LAB** (AUTOM series) — provider-lifecycle depth for the same `dynatrace_iam_*` resources: permission-DSL discovery, bulk export/import of an existing account, deprecated arguments, CI/CD integration
- Repo: [Terraform-IAM-MZ-Boundries](#) (schema-verified against provider v1.100.0)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.